

Generator Dokumentasi Kode Program Python dengan Memanfaatkan Large Language Model dan Prompt Techniques

Richard Rafer Guy, *Departemen Informatika Institut Sains dan Teknologi Terpadu Surabaya, Gunawan, Departemen Informatika Institut Sains dan Teknologi Terpadu Surabaya*

Abstrak—Dokumentasi kode merupakan elemen vital dalam pengembangan perangkat lunak untuk memastikan pemahaman dan keberlanjutan sistem. Namun, proses pembuatan dokumentasi secara manual sering kali tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun Large Language Models (LLM) menawarkan potensi otomatisasi, pendekatan standar sering kali gagal menangkap konteks dan memetakan dependensi kode yang kompleks, sehingga menghasilkan dokumentasi yang kurang akurat. Sistem ini mengembangkan perangkat lunak generator dokumentasi kode program Python otomatis yang memanfaatkan LLM dan teknik prompt engineering untuk mengatasi tantangan tersebut. Sistem yang dibangun terdiri dari dua komponen utama, yaitu Module Navigator dan Document Generator. Module Navigator melakukan analisis statis menggunakan Abstract Syntax Tree (AST) untuk memetakan graf dependensi dan menentukan urutan pemrosesan yang logis. Sementara itu, Document Generator menggunakan arsitektur multi-modul yang mencakup Reader, Searcher, Writer, dan Verifier untuk menyusun narasi dokumentasi. Pendekatan ini memastikan bahwa setiap dokumentasi disusun berdasarkan konteks yang relevan, melalui pencarian informasi internal dan eksternal yang terarah, serta melalui proses verifikasi yang ketat. Evaluasi dilakukan terhadap berbagai repositori open-source dengan tingkat kompleksitas yang beragam menggunakan metrik Completeness, Helpfulness, dan Truthfulness. Hasil pengujian menunjukkan bahwa sistem mampu menghasilkan dokumentasi dengan tingkat kelengkapan struktur dan akurasi faktual yang tinggi. Dalam evaluasi komparatif, metode yang diusulkan terbukti lebih unggul dibandingkan pendekatan vanilla prompting dalam aspek kebermanfaatan konten. Selain itu, evaluasi yang melibatkan pengembang perangkat lunak juga menunjukkan tingkat kepuasan yang positif, membuktikan bahwa pendekatan terstruktur dengan manajemen konteks efektif dalam menghasilkan dokumentasi yang komprehensif, akurat, dan bermanfaat bagi pengembang.

Kata Kunci—*Abstract Syntax Tree, Code Documentation, Large Language Models, Prompt Engineering, Python.*

I. PENDAHULUAN

Dokumentasi kode sumber (source code) merupakan artefak fundamental dalam siklus hidup pengembangan perangkat lunak yang berfungsi untuk mendeskripsikan fungsionalitas, tujuan, dan kontrak penggunaan suatu kode melalui teks penjelasan ataupun komentar [1]. Ketersediaan dokumentasi yang komprehensif sangat krusial untuk memfasilitasi pemeliharaan sistem, transfer pengetahuan, dan onboarding anggota tim baru. Namun, dalam praktiknya, pembuatan dokumentasi secara manual sering kali terabaikan karena prosesnya yang memakan waktu, tidak efisien, dan rentan menghasilkan output yang tidak terstruktur atau keliru seiring dengan evolusi kode yang cepat [2].

Kemajuan teknologi Large Language Models (LLM) menawarkan peluang signifikan untuk mengotomatisasi proses ini. Meskipun demikian, pendekatan standar yang mengandalkan vanilla prompting, yaitu memberikan instruksi langsung ke LLM tanpa manajemen konteks, sering kali menghadapi keterbatasan serius. LLM cenderung menghasilkan dokumentasi yang dangkal, tidak lengkap, atau bahkan mengalami halusinasi ketika dihadapkan pada basis kode yang kompleks dengan ketergantungan antar-modul yang rumit [2]. Kelemahan ini bersumber dari kesulitan model dalam memahami konteks proyek secara utuh dan memetakan graf dependensi secara mandiri dalam satu kali pemrosesan.

Untuk mengatasi tantangan tersebut, penelitian ini mengusulkan sebuah sistem generator dokumentasi otomatis yang dirancang dengan pendekatan dependency-aware. Sistem ini mengintegrasikan analisis statis menggunakan Abstract Syntax Tree (AST) dengan kemampuan generatif LLM melalui arsitektur multi-modul yang terorkestrasi. Berbeda dengan pendekatan konvensional, sistem ini memetakan informasi dan hubungan antar komponen kode terlebih dahulu untuk memastikan bahwa setiap proses generasi dokumentasi didukung oleh konteks yang relevan dan akurat.

Tujuan utama dari penelitian ini adalah membangun perangkat lunak yang mampu menghasilkan dokumentasi kode Python yang tidak hanya lengkap secara struktural, tetapi juga akurat secara faktual dan bermanfaat bagi pengembang. Artikel ini akan membahas metodologi pembangunan sistem, mulai dari pemetaan dependensi oleh Module Navigator hingga orkestrasi pembuatan konten oleh

02/12/2025

Richard Rafer Guy, Departemen Informatika, Institut Sains dan Teknologi Terpadu Surabaya, Surabaya, Jawa Timur, Indonesia (e-mail: richard.r22@mhs.istts.ac.id)

Gunawan, Departemen Informatika, Institut Sains dan Teknologi Terpadu Surabaya, Surabaya, Jawa Timur, Indonesia (e-mail: gunawan@stts.edu)

Document Generator, serta menyajikan hasil evaluasi kinerja sistem dibandingkan dengan metode baseline.

II. TINJAUAN PUSTAKA

Pengembangan sistem generator dokumentasi otomatis dalam penelitian ini dibangun di atas sejumlah konsep dan teknologi fundamental. Konsep dan teori tersebut akan dibahas, diawali dengan eksplorasi Large Language Model (LLM) dan Prompt Engineering sebagai teknologi utama pemrosesan teks. Selanjutnya, dibahas konsep mengenai Abstract Syntax Tree, dan konsep graf seperti Directed Acyclic Graph (DAG), algoritma PageRank, deteksi siklus, dan pengurutan topologis yang menjadi landasan analisis struktur kode sistem.

A. Large Language Model

Large Language Model (LLM) merupakan model komputasi berbasis jaringan saraf (neural network) dengan jumlah parameter masif yang dilatih menggunakan metode self-supervised learning pada korpus data teks yang sangat besar [3]. Karakteristik utama dari model ini adalah kemampuannya untuk memahami sintaksis, semantik, dan pengetahuan faktual guna menghasilkan teks yang koheren.

Fondasi teknis yang memungkinkan kapabilitas ini adalah arsitektur Transformer yang mengandalkan mekanisme self-attention untuk menangkap hubungan kontekstual yang kompleks antar kata tanpa terbatas oleh jarak sekuensial [4]. Selain kemampuan dasar, LLM modern menunjukkan kemampuan emergen seperti in-context learning, di mana model dapat beradaptasi dengan tugas baru hanya melalui instruksi atau contoh dalam prompt tanpa pembaruan parameter [3].

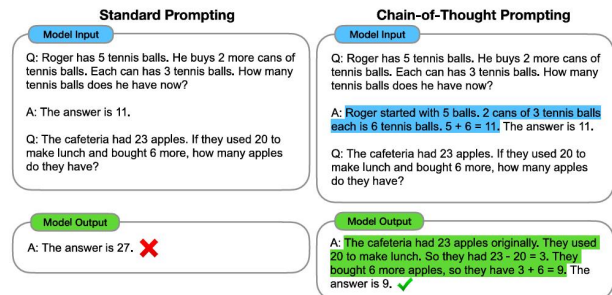
B. Prompt Engineering

Meskipun memiliki kapabilitas luas, kinerja LLM sangat bergantung pada kualitas instruksi yang diberikan, sehingga Prompt Engineering menjadi disiplin ilmu krusial yang berfokus pada perancangan dan optimisasi masukan (prompt). Sebuah prompt yang dirancang dengan buruk atau ambigu cenderung menghasilkan jawaban yang generik, tidak relevan, atau bahkan halusinasi fakta yang menyesatkan pengguna [5]. Oleh karena itu, teknik ini menekankan pada penyusunan instruksi yang spesifik, pemberian konteks yang memadai, dan penggunaan batasan yang jelas untuk memandu model agar menghasilkan luaran yang akurat dan sesuai dengan intensi pengguna. Metode interaksi paling mendasar dikenal sebagai Vanilla Prompting atau Standard Prompting, yaitu pendekatan memberikan instruksi langsung ke model tanpa strategi rekayasa tambahan. Meskipun sederhana, metode ini sering kali memiliki keterbatasan dalam menangani tugas yang membutuhkan pemahaman konteks mendalam [6].

1) Chain of Thought

Dalam konteks penyelesaian masalah yang kompleks, teknik Chain-of-Thought (CoT) diterapkan sebagai strategi prompting tingkat lanjut untuk meningkatkan kemampuan penalaran model secara signifikan. Metode ini bekerja

dengan cara memicu LLM untuk menjabarkan langkah-langkah berpikir logis atau rantai penalaran perantara secara eksplisit sebelum memberikan jawaban akhir [7]. Dengan menguraikan masalah besar menjadi serangkaian langkah logika yang lebih kecil, CoT terbukti efektif mengurangi kesalahan penalaran dan meningkatkan akurasi pada tugas-tugas yang membutuhkan pemikiran bertingkat dibandingkan dengan metode standard prompting.

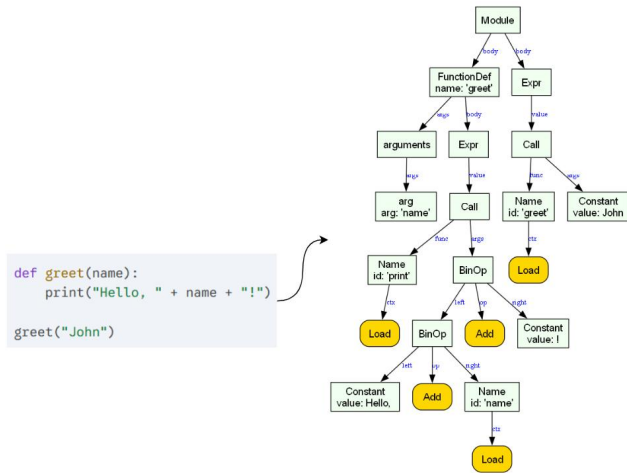


Gambar. 1. Perbandingan Standard Prompting dan CoT Prompting

C. Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah representasi hierarkis berbentuk pohon dari struktur sintaksis abstrak kode sumber yang ditulis dalam suatu bahasa pemrograman. Berbeda dengan parse tree yang memuat seluruh detail sintaksis termasuk tanda baca dan elemen tata bahasa, AST menyederhanakan representasi dengan hanya menyimpan informasi struktural dan konten yang penting untuk analisis program. Struktur ini memungkinkan algoritma komputer untuk membedah dan memahami logika program tanpa terganggu oleh detail sintaksis yang tidak relevan.

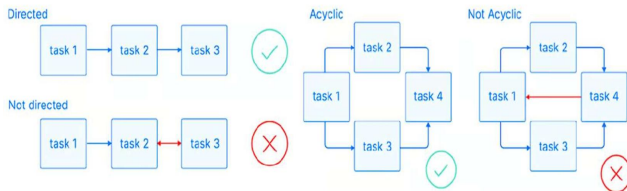
Dalam lingkungan pengembangan Python, modul AST memungkinkan penguraian kode sumber menjadi rangkaian simpul-simpul objek yang merepresentasikan konstruksi kode seperti fungsi, kelas, dan variabel. Kemampuan untuk menelusuri (traverse) pohon ini secara programatik memungkinkan sistem untuk mengekstrak informasi presisi mengenai komponen kode, seperti nama kelas, argumen fungsi, dan hubungan pewarisan secara statis. Hal ini menjadi landasan utama bagi Module Navigator dalam memetakan struktur repositori tanpa perlu mengeksekusi kode tersebut secara dinamis.



Gambar. 2. Representasi Abstract Syntax Tree pada Kode Python Sederhana

D. Directed Acyclic Graph

Untuk memodelkan struktur ketergantungan kode secara matematis, penelitian ini memanfaatkan konsep Directed Acyclic Graph (DAG), yaitu struktur graf berarah yang terdiri dari simpul (nodes) dan sisi (edges). Dalam konteks ini, simpul merepresentasikan entitas kode seperti fungsi atau kelas, sedangkan sisi berarah menggambarkan hubungan ketergantungan di mana satu komponen membutuhkan komponen lain untuk beroperasi. Sifat utama dari DAG adalah ketiadaan siklus, yang berarti tidak terdapat jalur yang memungkinkan penelusuran kembali ke simpul awal setelah meninggalkannya [8].



Gambar. 3. Ilustrasi Struktur DAG dan Perbedaannya dengan Grafik Tidak Terarah dan Siklik

Sifat asiklik ini sangat fundamental dalam manajemen alur kerja (workflow) dan analisis dependensi perangkat lunak untuk menjamin urutan eksekusi yang valid. Dengan memastikan struktur graf bebas dari siklus, sistem dapat menentukan urutan pemrosesan komponen kode secara linier dan logis. Hal ini mencegah risiko terjadinya deadlock atau perulangan tak terbatas (infinite loops) yang dapat melumpuhkan proses analisis, serta menjamin bahwa setiap dependensi dapat diselesaikan sebelum komponen induknya diproses.

E. Page Rank

Algoritma PageRank, yang awalnya dikembangkan untuk memeringkat halaman web berdasarkan struktur tautan, diadaptasi dalam penelitian ini untuk mengukur tingkat kepentingan sebuah komponen kode. Dalam konteks analisis statis, komponen kode diperlakukan sebagai simpul dan

hubungan ketergantungan diperlakukan sebagai tautan yang memberikan vote atau bobot. Prinsip dasarnya adalah bahwa sebuah komponen dianggap penting jika ia menjadi dependensi bagi banyak komponen lain, atau jika ia diandalkan oleh komponen lain yang juga memiliki tingkat kepentingan tinggi [9].

Penerapan algoritma ini memungkinkan sistem untuk secara otomatis mengidentifikasi komponen-komponen krusial yang menjadi tulang punggung arsitektur aplikasi. Skor kepentingan yang dihasilkan kemudian digunakan sebagai heuristik strategis dalam manajemen konteks pada modul Searcher. Ketika kapasitas token LLM terbatas, sistem akan memprioritaskan penyertaan informasi dari komponen dengan skor PageRank tinggi, memastikan bahwa dokumentasi yang dihasilkan didukung oleh konteks yang paling relevan dan berpengaruh.

F. Cycle Detection

Deteksi siklus (cycle detection) merupakan konsep fundamental dalam teori graf berarah untuk mengidentifikasi adanya ketergantungan melingkar yang dapat menghambat proses eksekusi linear. Dalam pengembangan perangkat lunak yang kompleks, sering kali terjadi situasi di mana komponen A bergantung pada komponen B, dan komponen B secara tidak langsung kembali bergantung pada komponen A. Keberadaan siklus semacam ini harus dideteksi dan diselesaikan terlebih dahulu agar struktur graf dapat diubah menjadi DAG yang valid untuk pengurutan.

Untuk menangani masalah ini, sistem menerapkan algoritma Tarjan yang berbasis pada penelusuran depth-first search (DFS) guna mengidentifikasi Strongly Connected Components (SCC). Komponen-komponen yang teridentifikasi berada dalam satu siklus ketergantungan kemudian dikelompokkan atau dikondensasi menjadi satu simpul super tunggal dalam graf yang baru. Langkah kondensasi ini memastikan bahwa graf hasil akhir tetap mempertahankan sifat asiklik sehingga dapat diproses lebih lanjut tanpa terjebak dalam referensi melingkar [10].

G. Topological Sorting

Topological sorting adalah metode pengurutan linear untuk simpul-simpul dalam sebuah DAG, yang menjamin ketaatan terhadap prasyarat ketergantungan. Prinsip algoritma ini menyatakan bahwa untuk setiap sisi berarah dari simpul u ke v , simpul u (sebagai dependensi) harus selalu muncul sebelum simpul v (sebagai komponen yang bergantung) dalam urutan tersebut. Algoritma ini hanya dapat diterapkan pada graf yang sudah dipastikan bebas dari siklus melalui proses deteksi dan kondensasi sebelumnya.

Dalam sistem generator dokumentasi ini, topological sorting diterapkan untuk menyusun antrian pemrosesan komponen kode secara otomatis. Dengan mengikuti urutan topologis, sistem menjamin bahwa setiap komponen kode hanya akan diproses oleh Document Generator setelah seluruh komponen yang menjadi dependensinya telah selesai didokumentasikan. Hal ini memungkinkan ketersediaan konteks yang lengkap dan akurat saat mendokumentasikan komponen di tingkat yang lebih tinggi, sehingga

meningkatkan kualitas dan koherensi dokumentasi secara keseluruhan [11].

III. METODE DAN INTI PENELITIAN

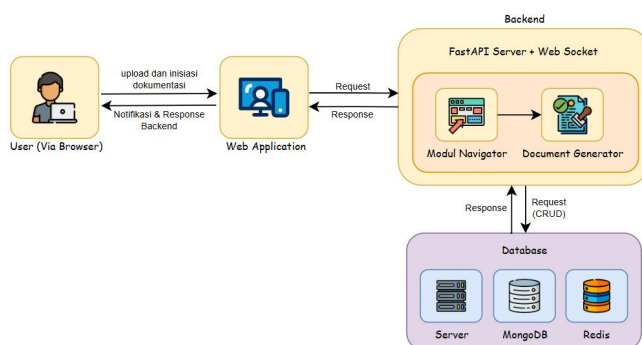
Metodologi yang diterapkan dalam penelitian ini berfokus pada pengembangan sistem perangkat lunak yang mengintegrasikan teknik analisis statis kode (static analysis) dengan kemampuan generatif Large Language Model (LLM). Pendekatan ini dirancang untuk mengatasi keterbatasan pemahaman konteks pada model bahasa standar dengan cara memisahkan proses pemetaan struktur kode dari proses penulisan dokumentasi. Sistem dibangun melalui tahapan perancangan arsitektur modular, pengembangan algoritma pemetaan dependensi, dan orkestrasi alur kerja multi-modul untuk menghasilkan dokumentasi yang akurat dan terstruktur secara otomatis.

A. Arsitektur Sistem

Sistem dirancang menggunakan pendekatan arsitektur modular yang memisahkan tanggung jawab antara antarmuka pengguna (frontend), logika pemrosesan (backend), dan lapisan data (database). Pemisahan ini bertujuan untuk menciptakan sistem yang terorganisir, skalabel, dan mudah dipelihara..

Alur interaksi sistem dimulai dari pengguna yang mengunggah repositori kode (berkas .zip) melalui aplikasi web yang dibangun menggunakan kerangka kerja React JS. Aplikasi web kemudian meneruskan permintaan ke backend untuk diproses secara asinkron. Mengingat proses analisis dan generasi dokumentasi memerlukan waktu komputasi yang signifikan, komunikasi antara frontend dan backend memanfaatkan protokol Web Socket untuk mengirimkan notifikasi dan pembaruan status pemrosesan secara real-time kepada pengguna.

Komponen backend berfungsi sebagai pusat pemrosesan utama yang dibangun di atas kerangka kerja FastAPI dengan bahasa pemrograman Python. Di dalam backend, terdapat dua modul pemrosesan inti yang bekerja secara berurutan, yaitu Module Navigator dan Document Generator. Module Navigator bertanggung jawab untuk melakukan analisis statis struktur kode dan memetakan dependensi, sedangkan Document Generator bertugas menyusun narasi dokumentasi dengan memanfaatkan Large Language Model (LLM).

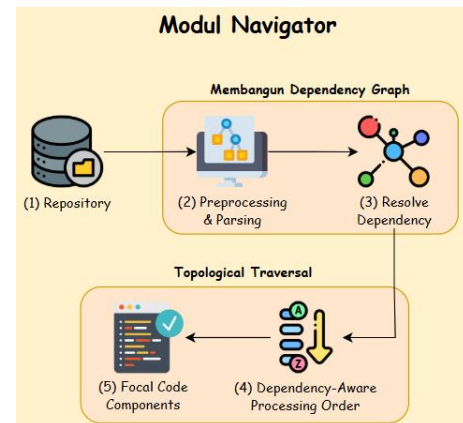


Gambar. 4. Arsitektur Keseluruhan Sistem

Untuk manajemen data, sistem menerapkan strategi penyimpanan hibrida yang mengintegrasikan penyimpanan server berbasis sistem berkas untuk menampung berkas mentah dan hasil ekstraksi sementara, basis data in-memory Redis untuk melacak status tugas (job state) secara real-time, serta MongoDB sebagai repositori persisten untuk menyimpan hasil akhir dokumentasi yang terstruktur .

B. Module Navigator

Module Navigator berfungsi sebagai mesin analisis statis yang bertugas mengubah repositori kode mentah menjadi informasi yang berharga, termasuk representasi graf terstruktur dan menentukan urutan pemrosesan yang logis. Alur kerja modul ini dirancang dalam dua fase utama, yaitu pembangunan graf dependensi (Dependency Graph Construction) dan penelusuran topologis (Topological Traversal).

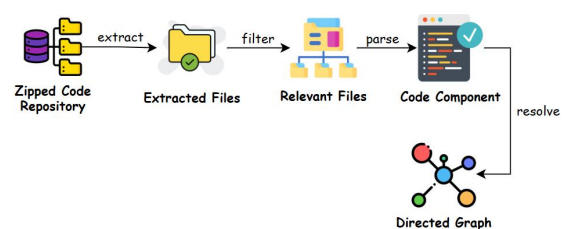


Gambar. 5. Ilustrasi Proses Modul Navigator

Sebagaimana diilustrasikan pada Gambar 5, transformasi repositori kode mentah menjadi daftar komponen terurut (Focal Code Components) melibatkan serangkaian tahapan analisis statis yang sistematis guna memastikan setiap dependensi terpetakan dengan akurat.

1) Preprocessing dan Parsing

Preprocessing dan Parsing merupakan tahap awal berfokus pada ekstraksi dan identifikasi entitas kode. Proses ini dimulai dengan mengekstrak berkas .zip, melakukan penyaringan (filtering) untuk mendapatkan berkas Python yang relevan, dan mengabaikan direktori yang tidak diperlukan (seperti testing atau konfigurasi lingkungan).



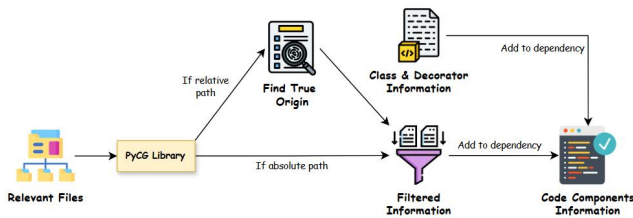
Gambar. 6. Ilustrasi Proses Directed Graph dari Ketergantungan Kode

Setelah berkas relevan didapatkan, sistem melakukan

parsing menggunakan bantuan Abstract Syntax Tree (AST). Pada tahap ini, setiap berkas dipindai untuk mengidentifikasi definisi kelas, fungsi, dan metode. Hasilnya disimpan sebagai objek komponen kode (Code Component) yang memuat metadata esensial seperti nama, lokasi baris, dan signatur kode, namun belum memuat hubungan ketergantungan antar-komponen.

2) Resolve Dependency

Setelah inventarisasi komponen selesai, langkah krusial berikutnya adalah memetakan hubungan ketergantungan (dependency resolution). Sistem memanfaatkan analisa Abstract Syntax Tree (AST) dari kode dan juga dibantu dengan pustaka analisis statis PyCG untuk melacak hubungan ketergantungan yang dapat terjadi karena beberapa keadaan seperti, pemanggilan fungsi (function calls) dan instansiasi kelas, inheritance, dan lainnya.

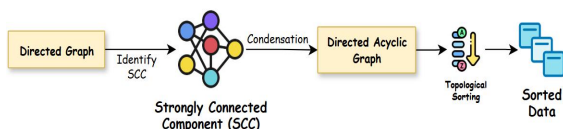


Gambar. 7. Alur Proses Pembangunan Dependency

Gambar 7 memperlihatkan logika resolusi yang diterapkan. Sistem untuk menangani tantangan jalur impor yang bervariasi (baik absolut maupun relative) dengan melacak asal definisi kode yang sebenarnya. Jika jalur bersifat relatif, sistem melakukan penelusuran balik untuk menemukan definisi aslinya. Informasi hubungan yang telah diverifikasi kemudian digunakan untuk membangun graf berarah (Directed Graph) menggunakan pustaka NetworkX, yang menyediakan struktur data efisien untuk merepresentasikan jaringan dependensi kompleks tersebut [12].

3) Dependency Aware Processing Order

Tantangan utama dalam pemetaan dependensi adalah keberadaan siklus (circular dependencies) yang umum terjadi dalam proyek perangkat lunak kompleks. Siklus ini dapat menghambat penentuan urutan eksekusi yang linear. Tanpa penyelesaian terhadap siklus tersebut, struktur graf tidak dapat memenuhi kriteria Directed Acyclic Graph (DAG) yang menjadi prasyarat mutlak agar algoritma pengurutan dapat dijalankan.



Gambar. 8. Alur Transformasi Graph dan Pengurutan Topologis

Untuk mengatasi keberadaan siklus pada sebuah graf, akan diterapkan serangkaian langkah untuk mengatasi hal tersebut seperti yang terdapat pada Gambar 8, sistem menerapkan algoritma Tarjan guna mendeteksi Strongly Connected Components (SCC). Komponen-komponen yang terlibat

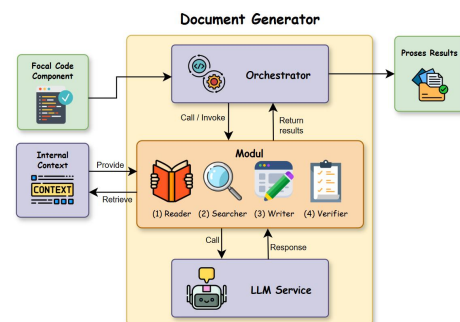
dalam siklus dikondensasi menjadi satu simpul super, mentransformasi graf menjadi Directed Acyclic Graph (DAG) yang valid. Setelah graf bebas siklus, algoritma Topological Sorting diterapkan untuk menyusun antrian pemrosesan akhir.

4) Focal Code Components

Focal Code Components merupakan sebuah Kumpulan informasi mengenai sebuah komponen kode yang telah teridentifikasi pada pemrosesan Module Navigator. Hal ini akan menjadi salah satu hasil utama dari Modul Navigator, karena ketika Document Generator ingin memproses atau mendapatkan konteks informasi mengenai sebuah komponen kode, termasuk hubungan ketergantungan dari sebuah komponen kode dengan komponen lainnya akan didapatkan melalui data tersebut.

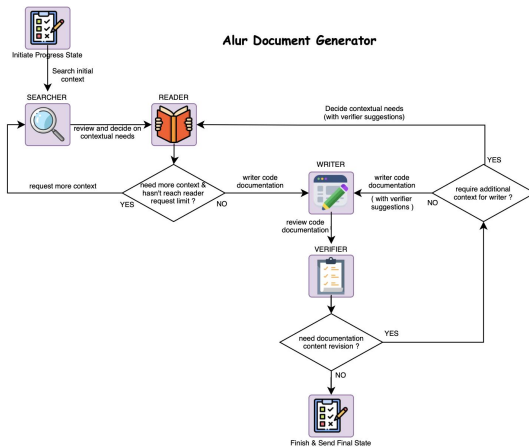
C. Document Generator

Arsitektur Document Generator dirancang untuk menangani kompleksitas pembuatan dokumentasi kode, di mana komponen ini bekerja setelah Module Navigator selesai dengan menerima Focal Code Component sebagai input utama. Tujuan utama arsitektur ini adalah menjalankan rangkaian proses kognitif terstruktur untuk menghasilkan Code Documentation yang relevan, didukung oleh Internal Context yang berisi informasi lengkap mengenai basis kode. Secara konseptual, arsitektur ini terbagi menjadi tiga lapisan utama, yaitu Orchestrator sebagai pengendali pusat di lapisan teratas, blok Modul yang berisi empat komponen spesialis (Reader, Searcher, Writer, Verifier) yang diturunkan dari BaseAgent, serta LLM Service di lapisan terbawah yang menghubungkan sistem dengan model bahasa melalui API.



Gambar. 9. Arsitektur Document Generator

Mekanisme interaksi antar komponen diatur secara hierarkis, di mana Orchestrator menginisiasi alur kerja dan mendelegasikan tugas kepada modul-modul spesialis secara berurutan tanpa berinteraksi langsung dengan LLM. Modul-modul tersebut bertanggung jawab atas tugas spesifik dan dapat memanggil LLM Service untuk kemampuan pemrosesan bahasa atau mengambil informasi tambahan dari Internal Context, sebuah aktivitas yang utamanya dilakukan oleh Searcher. Proses ini diakhiri dengan penyusunan dokumen lengkap, di mana hasil akhirnya mencakup informasi mengenai state pemrosesan terakhir, hasil dokumentasi, serta perhitungan total penggunaan token LLM selama eksekusi berlangsung.



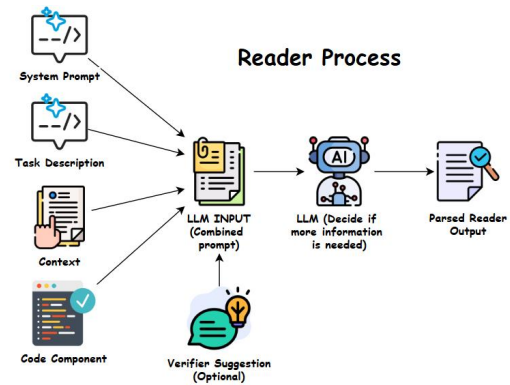
Gambar. 10. Alur Pemrosesan Orchestrator pada Document Generator

Alur pemrosesan dalam Document Generator dikendalikan sepenuhnya oleh Orchestrator sebagai siklus kognitif dinamis yang mengelola urutan eksekusi dan aliran data antar modul secara iteratif. Proses ini dimulai ketika Orchestrator menginisiasi state baru setelah menerima komponen kode (focal code), lalu memanggil Searcher untuk mengumpulkan konteks awal yang kemudian dievaluasi kritis oleh Reader. Jika Reader menilai informasi belum cukup, siklus pencarian tambahan ke Searcher akan dipicu berulang kali hingga konteks dianggap memadai, atau ketika mencapai batas iterasi yang ditentukan, sebelum akhirnya diserahkan kepada Writer untuk memulai tahap penulisan draf.

Setelah draf dokumentasi dihasilkan, Verifier melakukan penjaminan kualitas dengan kemampuan mengembalikan draf ke Writer untuk perbaikan konten atau ke Reader jika ditemukan kekurangan konteks mendasar. Proses dianggap selesai ketika Verifier menyetujui hasil akhir tanpa revisi lebih lanjut, atau batas penolakan draf oleh verifier telah tercapai, di mana Orchestrator kemudian mengirimkan dokumentasi final beserta metrik pemrosesan seperti penggunaan token. Efisiensi seluruh alur kerja ini dijaga melalui parameter konfigurasi yang menetapkan batas maksimum iterasi dan jumlah token konteks untuk mencegah perulangan tanpa henti.

1) Reader

Modul Reader berfungsi sebagai gerbang strategis yang bertugas menganalisis kecukupan informasi sebelum proses penulisan dimulai, memastikan konteks yang tersedia sudah memadai untuk menghasilkan dokumentasi berkualitas. Modul ini beroperasi di awal siklus dengan mengevaluasi komponen kode dan konteks awal. Jika informasi dinilai cukup, alur berlanjut ke Writer, namun jika kurang, Reader akan memicu siklus pengayaan ke Searcher untuk mencegah risiko halusinasi akibat informasi yang dangkal. Dalam peran ini, Reader tidak hanya mendeteksi kekurangan tetapi juga merumuskan permintaan spesifik, baik berupa ekspansi komponen internal maupun kueri eksternal, untuk mengarahkan strategi pengumpulan data Searcher secara presisi.

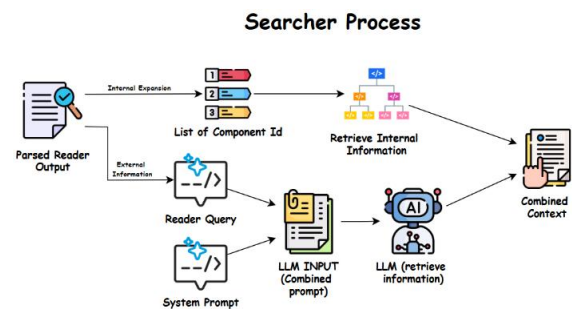


Gambar. 11. Alur Pemrosesan pada Reader

Alur pemrosesan internal Reader, sebagaimana diilustrasikan pada Gambar 11, diawali dengan konstruksi prompt komprehensif yang mengagregasi input statis seperti System Prompt dan Task Description dengan data dinamis dari AgentState. Data dinamis ini mencakup konteks dari Searcher, komponen kode yang dianalisis, serta saran opsional dari Verifier jika terjadi siklus umpan balik sebelumnya. Gabungan informasi tersebut kemudian diserahkan kepada LLM untuk diproses menjadi output terstruktur yang merepresentasikan keputusan akhir mengenai apakah informasi tambahan diperlukan untuk melanjutkan proses dokumentasi.

2) Searcher

Modul Searcher bertindak sebagai lengan eksekusi pengumpul data yang bertugas mengagregasi informasi dari sumber internal maupun eksternal untuk memenuhi permintaan Reader dan menyediakan paket konteks bagi Writer. Strategi pencarian internal dilakukan dengan menelusuri graf dependensi dari Module Navigator secara terprogram, sedangkan pencarian eksternal memanfaatkan kemampuan LLM untuk menjelaskan konsep yang tidak dapat disimpulkan hanya dari kode. Selain itu, Searcher beroperasi secara proaktif di awal siklus dengan mengumpulkan konteks dasar, di mana prioritas pengambilan informasi dependensi ditentukan berdasarkan skor PageRank untuk memastikan relevansi data yang paling berpengaruh disajikan sejak dini.



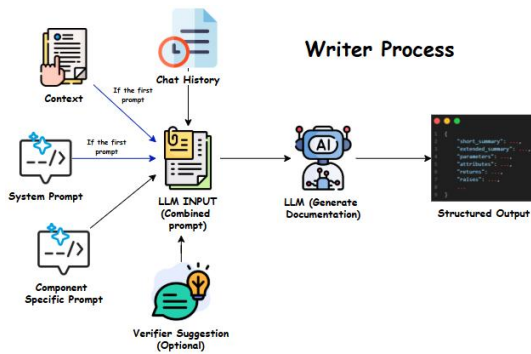
Gambar. 12. Alur Pemrosesan Searcher untuk Menangani Permintaan Searcher

Sebagaimana diilustrasikan pada Gambar 5.5, alur kerja Searcher merespons parsed output dari Reader dengan mengeksekusi ekspansi internal dan pencarian eksternal, lalu menggabungkan hasilnya menjadi Combined Context yang

utuh. Modul ini juga bertanggung jawab atas manajemen konteks cerdas untuk mematuhi batasan token, di mana strategi pemotongan bertingkat diterapkan jika total ukuran data melebihi konfigurasi. Dalam skenario tersebut, informasi yang dianggap kurang krusial, seperti konteks eksternal atau dependensi dengan skor PageRank terendah, akan dipangkas terlebih dahulu untuk menjaga efisiensi operasional tanpa mengorbankan informasi inti.

3) Writer

Modul Writer berfungsi sebagai komponen sentral untuk sintesis konten kreatif yang bertugas mengubah gabungan komponen kode dan paket konteks menjadi draf dokumentasi yang koheren dan terstruktur. Berbeda dengan modul analisis sebelumnya, Writer berfokus eksklusif pada generasi narasi dan diaktifkan oleh Orchestrator setelah siklus pengumpulan informasi oleh Reader dan Searcher selesai. Modul ini tidak bekerja dalam satu kali eksekusi, melainkan dirancang dalam siklus umpan balik iteratif dengan modul Verifier, di mana output yang dihasilkan diperlakukan sebagai draf yang akan dievaluasi dan direvisi berulang kali untuk mencapai standar kualitas tinggi.



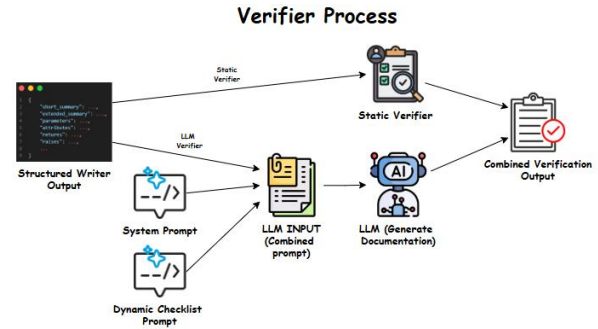
Gambar. 13. Ilustrasi Pemrosesan Writer

Sebagaimana diilustrasikan pada Gambar 13, alur pemrosesan internal Writer berpusat pada konstruksi prompt dinamis yang menggabungkan System Prompt, instruksi spesifik komponen, dan konteks untuk menghasilkan output JSON terstruktur demi konsistensi hasil. Dalam skenario revisi, alur kerja menjadi adaptif dengan memanfaatkan riwayat percakapan (Chat History) yang memuat draf sebelumnya dan saran perbaikan dari Verifier, alih-alih hanya mengandalkan konteks awal. Mekanisme ini memungkinkan Writer melakukan perbaikan terarah secara langsung kepada LLM, terus menyempurnakan dokumentasi hingga memenuhi kriteria validasi atau mencapai batas iterasi yang ditentukan.

4) Verifier

Modul Verifier berfungsi sebagai gerbang penjamin kualitas akhir yang bertugas mengevaluasi draf dokumentasi dari Writer untuk memastikan kesesuaian hasil sebelum diserahkan kepada pengguna. Keunggulan utama modul ini terletak pada kemampuannya mengimplementasikan siklus umpan balik cerdas, di mana ia tidak hanya menentukan kelulusan draf tetapi juga memberikan saran konstruktif yang dapat ditindaklanjuti untuk perbaikan. Umpan balik tersebut

digunakan oleh Orchestrator untuk mengarahkan revisi ke modul yang relevan, memungkinkan kontrol alur iterasi yang dinamis demi penyempurnaan konten secara progresif.

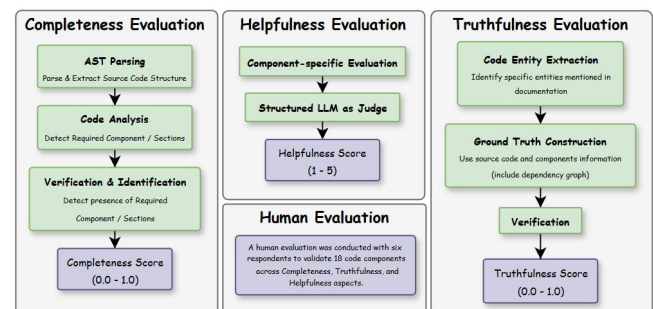


Gambar. 14. Ilustrasi Proses Verifikasi oleh Verifier

Sebagaimana diilustrasikan pada Gambar 14, Verifier mengadopsi pendekatan hibrida yang menggabungkan verifikasi statis berbasis Abstract Syntax Tree (AST) untuk pemeriksaan akurasi faktual deterministik seperti tipe data dan parameter, dengan verifikasi berbasis LLM untuk evaluasi semantik. Proses verifikasi LLM memanfaatkan System Prompt dan Dynamic Checklist untuk menilai aspek subjektif seperti kualitas deskripsi dan kepatuhan gaya penulisan. Hasil dari kedua metode tersebut kemudian dikonsolidasikan menjadi laporan gabungan berisi keputusan dan saran perbaikan yang menjadi landasan bagi Orchestrator dalam menentukan langkah selanjutnya.

IV. METRIK EVALUASI

Evaluasi kualitas sistem dilakukan secara komprehensif mencakup aspek struktural, konten, dan akurasi faktual menggunakan empat metrik utama, yaitu Completeness, Helpfulness, Truthfulness, serta Human Evaluation. Pendekatan ini bertujuan untuk mengukur performa otomatis sekaligus memvalidasi hasil melalui perspektif pengembang nyata, dengan rincian definisi dan implementasi teknis pengukuran yang akan diuraikan selanjutnya.



Gambar. 15. Metrik Evaluasi Kualitas Sistem

A. Completeness

Metrik Completeness berfungsi untuk mengevaluasi sejauh mana dokumentasi yang dihasilkan mematuhi standar struktur dan mencakup komponen esensial seperti ringkasan, deskripsi parameter, dan nilai kembalian sesuai dengan karakteristik elemen kode. Penilaian ini bersifat dinamis, di

mana keberadaan elemen yang wajib ada disesuaikan dengan logika kode spesifik, misalnya fungsi tanpa parameter tidak mewajibkan bagian parameter dalam dokumentasinya. Pendekatan ini memberikan indikasi yang jelas dan objektif mengenai kepatuhan struktural dokumentasi terhadap persyaratan kelengkapan yang diharapkan.

Pengukuran kuantitatif dilakukan melalui mekanisme otomatis berbasis analisis Abstract Syntax Tree (AST) yang memindai kode sumber untuk mengidentifikasi komponen dan mendeteksi bagian dokumentasi yang wajib tersedia berdasarkan signatur dan tubuh kode. Proses ini dilanjutkan dengan tahap verifikasi, di mana sistem mencocokkan keberadaan bagian standar dalam dokumentasi yang dihasilkan, dengan persyaratan yang telah dipetakan. Skor akhir completeness kemudian dihitung sebagai rasio proporsional antara komponen wajib yang ditemukan terhadap total persyaratan, menghasilkan nilai ternormalisasi antara 0.0 hingga 1.0 yang merepresentasikan kualitas struktural dokumen tersebut.

B. Helpfulness

Metrik Helpfulness dirancang untuk mengevaluasi kualitas semantik dan utilitas praktis dokumentasi, memastikan bahwa informasi yang disajikan memberikan nilai tambah di luar sekadar repetisi sintaksis kode. Evaluasi ini berfokus pada kejelasan, kedalaman deskripsi, dan kebermanfaatan parameter agar dokumentasi dapat berfungsi efektif sebagai panduan pemahaman bagi pengembang. Mengingat sifat subjektif dari penilaian ini, sistem mengadopsi pendekatan LLM-as-a-judge yang memecah evaluasi menjadi komponen spesifik seperti ringkasan dan deskripsi parameter untuk dinilai secara independen.

Implementasi teknis pengukuran ini diperkuat dengan strategi structured prompting komprehensif yang mencakup rubrik penilaian eksplisit pada skala 1 hingga 5 serta penyertaan few-shot examples sebagai panduan bagi LLM. Selain itu, instruksi Chain-of-Thought diterapkan untuk memandu model melakukan analisis mendalam dan menyusun penalaran logis sebelum memberikan skor akhir. Hasil evaluasi kemudian diformat menggunakan tag XML standar untuk memastikan bahwa skor yang dihasilkan dapat diproses secara otomatis dan konsisten oleh sistem.

C. Truthfulness

Metrik Truthfulness dirancang untuk mengukur akurasi faktual dokumentasi dan memitigasi risiko halusinasi, yaitu kondisi di mana LLM merujuk pada entitas kode yang tidak eksis akibat kurangnya pengetahuan bawaan mengenai basis kode privat. Evaluasi ini bertujuan memverifikasi bahwa setiap entitas seperti kelas atau fungsi yang disebutkan dalam narasi benar-benar dapat ditemukan dalam repositori target, sehingga mencegah informasi yang menyesatkan bagi pengembang. Proses evaluasi dimulai dengan tahap Code Entity Extraction, di mana sistem menggunakan LLM dengan prompt khusus untuk mengidentifikasi penyebutan komponen kode internal dalam dokumentasi, sambil secara eksplisit mengecualikan kata kunci standar bahasa pemrograman agar fokus tetap pada referensi proyek.

Tahap selanjutnya adalah Ground Truth Construction yang memanfaatkan data struktural dari Module Navigator, termasuk informasi komponen dan graf dependensi, sebagai acuan kebenaran mutlak mengenai seluruh komponen yang valid dalam repositori. Proses diakhiri dengan verifikasi, di mana setiap entitas yang diekstrak dicocokkan dengan ground truth melalui pemeriksaan nama langsung atau penelusuran atribut menggunakan Abstract Syntax Tree (AST). Skor akhir Truthfulness kemudian dikuantifikasi menggunakan Existence Ratio, yaitu proporsi entitas valid dibandingkan total entitas yang teridentifikasi, menghasilkan nilai antara 0.0 hingga 1.0 yang mencerminkan tingkat kepercayaan faktual dokumen tersebut.

D. Human Evaluation

Evaluasi kualitas dokumentasi diperkuat dengan tahapan Human Evaluation yang melibatkan partisipasi aktif dari enam pengembang perangkat lunak berpengalaman untuk memvalidasi metrik otomatis dari perspektif pengguna nyata. Proses ini bertujuan untuk menangkap penilaian subjektif mengenai nilai praktis, keterbacaan, dan relevansi dokumentasi terhadap standar industri saat ini. Mekanisme penilaian dilakukan melalui kuesioner terstruktur yang mengukur tiga dimensi utama, yaitu Completeness untuk memvalidasi kelengkapan elemen struktural, Truthfulness untuk memeriksa akurasi faktual dan ketiadaan halusinasi, serta Helpfulness untuk menilai efektivitas penyampaian informasi dalam membantu pemahaman kode.

Untuk menjamin validitas dan objektivitas data, setiap komponen kode dinilai secara independen oleh responden yang terdiri dari kombinasi mahasiswa tingkat akhir yang sedang menjalani internship dan pengembang perangkat di industri. Sistem penilaian menggunakan skala linear untuk mengukur persepsi responden terhadap berbagai aspek, mulai dari kejelasan ringkasan hingga ketepatan deskripsi parameter teknis. Dengan metode ini, evaluasi tidak hanya mengandalkan algoritma, tetapi juga memberikan gambaran komprehensif mengenai seberapa sukses sistem dalam menghasilkan dokumentasi yang benar-benar bermanfaat dan dapat diandalkan dalam skenario pengembangan perangkat lunak yang sesungguhnya.

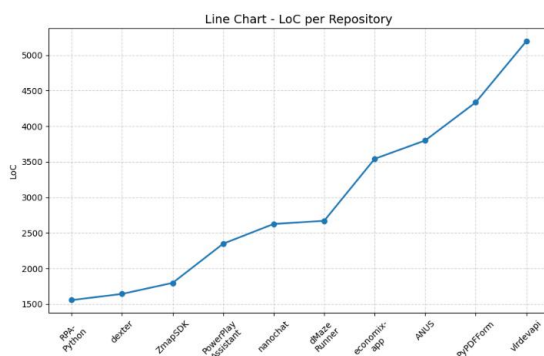
E. Uji Perbandingan Sistem terhadap Baseline pada Delapan Komponen Kode Sampel

Untuk memvalidasi keunggulan sistem yang dikembangkan, hasil dokumentasi dibandingkan secara langsung dengan pendekatan baseline berupa Vanilla Prompting atau Standard Prompting, yang mengandalkan kemampuan dasar Large Language Model (LLM) tanpa bantuan manajemen konteks eksternal maupun teknik penalaran canggih seperti Chain-of-Thought. Perbandingan ini melibatkan dua variasi prompt standar, variasi pertama menggunakan instruksi langsung tanpa contoh (zero-shot) yang meminta model menyusun dokumentasi lengkap berdasarkan kode sumber semata, dan variasi kedua menerapkan few-shot prompting dengan menyertakan contoh pasangan input-output untuk memberikan panduan format yang lebih jelas. Evaluasi terhadap baseline ini bertujuan

untuk memetakan secara jelas nilai tambah dari arsitektur sistem yang diusulkan dalam menangani kompleksitas kode yang sering kali gagal ditangkap oleh interaksi LLM konvensional.

V. DATASET UJI COBA

Pengujian sistem dilakukan menggunakan dataset utama yang terdiri dari sepuluh repositori proyek Python open-source terpilih yang merepresentasikan spektrum kompleksitas yang beragam. Distribusi ukuran kode dalam dataset ini dirancang untuk mencakup rentang target secara merata, mulai dari proyek terkecil dengan sekitar 1.000 Lines of Code (LoC) hingga mencapai puncaknya pada repositori vlrdevapi dengan jumlah 5.194 LoC.



Gambar. 16. Line Chart LoC per Repository

Selain metrik kuantitatif, dataset ini telah divalidasi memenuhi persyaratan kriteria seleksi yang mencakup variasi domain aplikasi yang luas, seperti pengembangan API menggunakan kerangka kerja Flask dan implementasi berbasis Python function. Daftar repositori yang digunakan juga mengonfirmasi keberadaan proyek yang berfokus pada domain Artificial Intelligence dan Machine Learning, memastikan sistem diuji terhadap berbagai karakteristik kode.

TABEL I
DAFTAR REPOSITORY UJI COBA

Repository	File Valid	LoC	Komponen
Vlrdevapi	22	5194	136
PyPDFForm	33	4336	148
ANUS	33	3800	165
Economix-App	1	3542	135
dMazeRunner	10	2670	139
Nanochat	17	2626	159
PowerPlayAssistant	40	2348	112
ZmapSDK	17	1797	110
Dexter	29	1642	104
RPA-Python	5	1555	78

File valid merupakan jumlah file relevan yang dapat didokumentasikan, sedangkan Komponen merupakan jumlah komponen kode yang ada.

A. Dataset Human Evaluation

Pada Human Evaluation, untuk menjamin evaluasi yang representatif terhadap berbagai skala proyek, pengujian melibatkan tiga repositori dengan tingkat kompleksitas Lines of Code (LoC) yang berbeda, yaitu RPA-Python (1.555 LoC)

untuk kategori rendah, Autonomous-Networked-Utility-System (3.800 LoC) untuk menengah, dan vlrdevapi (5.194 LoC) sebagai representasi kompleksitas tinggi. Sebanyak 18 komponen kode sampel dievaluasi, terdiri dari enam komponen per repositori yang dipilih secara strategis untuk mencakup rentang variasi struktural dari logika sederhana hingga kompleks dengan komposisi yang seimbang antara kelas dan fungsi, kecuali pada RPA-Python yang difokuskan pada fungsi karena keterbatasan struktur kelas. Pendekatan sampling ini dirancang untuk memvalidasi kinerja sistem secara komprehensif terhadap berbagai jenis entitas kode yang mungkin ditemui dalam pengembangan nyata.

TABEL II
INFORMASI DATA SAMPLE PADA HUMAN EVALUATION

Repository	Class	Function	Total
Vlrdevapi	3	3	6
ANUS	3	3	6
RPA-Python	0	6	6

Terdapat total 18 Komponen Kode yang digunakan pada Human Evaluation

B. Dataset Pengujian Sistem dan Baseline

Pengujian antara sistem dengan baseline dilaksanakan menggunakan sampel data yang bersumber dari repositori ZMapSDK. Pemilihan repositori ini didasarkan pada karakteristik ukurannya yang moderat serta keberagaman struktur kodenya yang mencakup implementasi pemrograman berorientasi objek maupun fungsional. Sebanyak delapan komponen kode dipilih sebagai subjek evaluasi utama untuk dibandingkan dengan metode baseline. Komposisi sampel tersebut dirancang secara merata, terdiri dari empat komponen bertipe kelas (Class) dan empat komponen bertipe fungsi atau metode (Function/Method), guna memastikan pengujian mencakup variasi pola penulisan kode yang representatif.

VI. HASIL UJI COBA DAN EVALUASI

Bagian ini memaparkan hasil evaluasi kinerja sistem generator dokumentasi berdasarkan rangkaian pengujian. Penilaian difokuskan pada pengukuran aspek kelengkapan struktur, akurasi faktual, dan nilai kebermanfaatan konten. Data berikut menunjukkan pencapaian sistem dalam memenuhi standar kualitas yang telah ditetapkan.

A. Hasil Uji Coba dan Evaluasi Otomatis

Hasil pengukuran metrik otomatis menunjukkan kinerja sistem yang sangat memuaskan di seluruh dimensi evaluasi kualitas. Pada aspek *Completeness*, sistem mencapai skor rata-rata 98,43%, dengan beberapa repositori seperti *vlrdevapi* mencapai nilai sempurna, yang mengindikasikan kemampuan andal dalam mematuhi standar struktur dokumentasi meskipun terdapat sedikit variasi pada penyertaan contoh penggunaan untuk komponen sederhana. Kinerja impresif juga terlihat pada metrik *Truthfulness* dengan skor rata-rata 96,64%, di mana sistem berhasil

memverifikasi validitas 3.711 dari 3.840 entitas yang diekstrak, membuktikan efektivitas arsitektur dalam meminimalkan risiko halusinasi faktual meskipun masih terdapat tantangan minor dalam pelacakan entitas granular yang kompleks.

Evaluasi kualitas semantik melalui metrik Helpfulness mencatatkan skor rata-rata keseluruhan 3,63 dari skala 5,00, dengan keunggulan spesifik pada penjelasan parameter dan deskripsi mendalam yang masing-masing memperoleh skor 3,88 dan 3,78. Meskipun skor ringkasan (summary) sedikit lebih rendah di angka 3,24 akibat preferensi model terhadap gaya penulisan yang sangat padat, hasil ini tetap menegaskan bahwa dokumentasi yang dihasilkan tidak hanya akurat secara teknis tetapi juga memberikan nilai tambah yang signifikan bagi pemahaman pengembang. Secara keseluruhan, integrasi ketiga metrik ini mengonfirmasi bahwa sistem mampu menghasilkan dokumentasi yang lengkap secara struktural, akurat secara faktual, dan bermanfaat secara semantik dalam berbagai skenario kode nyata.

TABEL III
HASIL UJI COBA OTOMATIS

Repository	Completeness	Helpfulness	Truthfulness
Vlrdevapi	100.00 %	3.72	98.00 %
PyPDFForm	99.26 %	3.52	91.44 %
ANUS	99.48 %	3.59	99.54 %
Economix-App	94.22 %	3.51	93.96 %
dMazeRunner	98.32 %	3.59	95.04 %
Nanochat	98.91 %	3.62	97.72 %
PowerPlayAssistant	94.91 %	3.58	97.21 %
ZmapSDK	99.85 %	3.74	100.00 %
Dexter	99.84 %	3.72	99.17 %
RPA-Python	99.49 %	3.75	97.54 %

Hasil analisis operasional sistem terhadap efisiensi biaya menunjukkan bahwa pengeluaran komputasi berbanding lurus dengan skala dan kompleksitas repositori, di mana proyek besar seperti Nanochat membutuhkan alokasi biaya token yang lebih tinggi. Variasi biaya ini sangat dipengaruhi oleh volume Lines of Code (LoC) yang meningkatkan konsumsi token input-output serta kompleksitas graf dependensi yang menuntut penyisipan konteks lebih padat oleh modul Searcher. Selain karakteristik kode, konfigurasi internal mengenai batas iterasi modul juga teridentifikasi sebagai faktor penentu dalam pengendalian efisiensi biaya akhir.

Dari segi durasi pemrosesan, sistem menunjukkan kinerja yang andal dengan rata-rata waktu penyelesaian sekitar satu menit per komponen, bahkan mencapai kecepatan di bawah 50 detik pada repositori tertentu seperti vlrdevapi. Meskipun kinerja umumnya stabil, terdapat anomali pada repositori dexter dengan durasi rata-rata 2 menit 11 detik yang teridentifikasi disebabkan oleh ketidakstabilan koneksi internet perangkat penguji. Hal ini menegaskan bahwa selain optimasi algoritma internal, stabilitas infrastruktur jaringan eksternal memegang peranan vital dalam menjaga konsistensi waktu pemrosesan sistem.

TABEL IV
BIAYA DAN WAKTU PEMROSESAN

Repository	Komponen	Biaya	Durasi
Vlrdevapi	136	Rp 72,453	01:55:33
PyPDFForm	148	Rp 80,253	02:28:04
ANUS	165	Rp 77,881	02:16:55
Economix-App	135	Rp 73,040	02:40:27
dMazeRunner	139	Rp 89,342	02:19:42
Nanochat	159	Rp 89,354	02:57:33
PowerPlayAssistant	112	Rp 61,633	01:39:48
ZmapSDK	110	Rp 72,335	01:50:57
Dexter	104	Rp 50,762	03:47:05
RPA-Python	78	Rp 53,334	01:26:32

Durasi yang dituliskan dalam format jam : menit : detik

B. Hasil Human Evaluation

Hasil Human Evaluation yang melibatkan enam pengembang aplikasi menunjukkan tingkat kepuasan yang sangat tinggi terhadap kualitas dokumentasi yang dihasilkan sistem, memvalidasi efektivitasnya dalam skenario penggunaan nyata. Pada aspek Completeness, mayoritas responden memberikan skor sempurna dengan rata-rata kumulatif 5,80 dari skala 6, menegaskan bahwa struktur dokumentasi dinilai utuh dan komprehensif. Demikian pula, aspek Truthfulness memperoleh skor rata-rata 5,78 dengan dominasi penilaian tertinggi, yang mengindikasikan bahwa para pengembang jarang menemukan ketidaksesuaian faktual atau halusinasi dalam deskripsi kode.

Evaluasi pada aspek Helpfulness mencatatkan skor rata-rata 5,74, di mana respons positif sangat dominan meskipun terdapat sedikit variasi nilai, mencerminkan bahwa dokumentasi tersebut dinilai efektif dalam membantu pemahaman konteks dan penggunaan kode. Secara keseluruhan, konsistensi skor tinggi di seluruh dimensi penilaian ini mengonfirmasi bahwa sistem tidak hanya unggul dalam metrik otomatis, tetapi juga dapat memenuhi standar ekspektasi industri dengan menghasilkan dokumentasi yang akurat secara teknis dan bernilai praktis bagi pengembang perangkat lunak.

TABEL V
HASIL HUMAN EVALUATION

Repository	Completeness	Helpfulness	Truthfulness
Vlrdevapi	5,88/6	5,6/6	5,9/6
ANUS	5,81/6	5,73/6	5,75/6
RPA-Python	5,71/6	5,6/6	5,69/6

Hasil uji coba tersebut dilakukan dengan menggunakan 3 repositori, dengan total 18 komponen kode yang digunakan

C. Hasil Pengujian Sistem dan Baseline

Hasil evaluasi komparatif pada sampel repositori ZmapSDK menunjukkan dominasi kinerja metode usulan dibandingkan pendekatan baseline (Gemini dan ChatGPT) di seluruh metrik pengujian. Keunggulan paling mencolok tercatat pada aspek Completeness, di mana metode usulan mencapai skor yang tinggi, jauh melampaui model pembanding yang hanya berkisar antara 43% hingga 72% meskipun telah diberikan panduan ringkas. Demikian pula pada aspek Truthfulness, metode usulan menjamin akurasi yang baik, sedikit mengungguli performa baseline yang

sudah cukup tinggi di kisaran 97-100%, membuktikan keandalan sistem dalam menjaga integritas fakta tanpa halusinasi.

Pada dimensi kualitas semantik, metode usulan mencatatkan skor Helpfulness sebesar 3,92, unggul signifikan dibandingkan skor tertinggi baseline yang hanya mencapai 3,24. Selisih nilai ini mengindikasikan bahwa strategi manajemen konteks yang diterapkan berhasil menghasilkan deskripsi yang lebih mendalam dan bernilai praktis, berbeda dengan baseline yang digunakan, yang cenderung menghasilkan narasi generik. Secara keseluruhan, data ini memvalidasi bahwa pendekatan terstruktur memberikan peningkatan kualitas yang nyata dibandingkan dengan baseline.

TABEL VI
HASIL PERBANDINGAN SISTEM DAN BASELINE

Repository	Completeness	Helpfulness	Truthfulness
Metode Usulan	100 %	100 %	3.92
Gemini 2.5 Pro FS	61.25 %	98 %	3.24
ChatGPT 5.1 FS	72.62 %	100 %	3.23
Gemini 2.5 Pro ZS	55.83 %	97 %	2.47
ChatGPT 5.1 ZS	43.75 %	100 %	2.60

D. Hasil Uji Coba Tambahan dengan Mistral

Uji coba validasi tambahan dilakukan pada tiga repositori terpilih menggunakan model bahasa alternatif dari Mistral AI (open-mixtral-8x22b dan mistral-large-2411) untuk menguji ketahanan arsitektur sistem pada lingkungan inferensi yang berbeda. Hasil eksperimen menunjukkan bahwa sistem mempertahankan stabilitas performa yang sangat baik pada aspek struktural dan faktual, dengan skor Completeness konsisten di atas 96% dan Truthfulness mencapai kisaran 97% hingga 99%, membuktikan bahwa mekanisme anti-halusinasi sistem bersifat robust dan tidak bergantung pada satu jenis model saja. Meskipun terdapat sedikit penurunan pada skor Helpfulness yang berkisar antara 3,15 hingga 3,21 yang dapat disebabkan karena perbedaan karakteristik model Mistral dalam Pembangunan narasi dibandingkan Gemini, secara keseluruhan sistem mampu menghasilkan dokumentasi yang valid secara teknis.

TABEL VII
DAFTAR REPOSITORY UJI COBA

Repository	Completeness	Helpfulness	Truthfulness
Vlrdevapi	96,75 %	3,15	98,56 %
ANUS	99,48 %	3,21	97,46 %
RPA-Python	99,68 %	3,21	99,37 %

E. Limitasi

Identifikasi batasan teknis sistem menyoroti kendala pada metode analisis statis yang bergantung pada pustaka PyCG dan Abstract Syntax Tree (AST). Optimasi pustaka yang terpaku pada lingkungan Python 3.9 berpotensi menyebabkan kegagalan deteksi pada sintaksis baru, sementara sifat analisis statis itu sendiri membatasi kemampuan sistem untuk melacak hubungan ketergantungan yang bersifat dinamis atau baru terbentuk saat runtime. Selain faktor internal, kinerja sistem juga sangat dipengaruhi oleh

variabel eksternal seperti stabilitas koneksi internet dan responsivitas layanan API model bahasa. Mengingat proses dokumentasi melibatkan pertukaran data yang intensif dengan pihak ketiga, gangguan pada infrastruktur jaringan ini berada di luar kendali sistem namun memegang peranan vital dalam menentukan kelancaran dan kecepatan penyelesaian tugas.

VII. KESIMPULAN

Penelitian ini berhasil mengembangkan sistem generator dokumentasi kode Python yang terbukti efektif menghasilkan luaran yang komprehensif dan akurat secara faktual. Hasil pengujian menunjukkan kinerja sistem yang superior dengan skor Completeness mencapai 98,43% dan Truthfulness sebesar 96,64%, mengindikasikan kemampuan arsitektur Module Navigator dan Document Generator dalam memetakan struktur kode serta meminimalkan risiko halusinasi informasi. Keunggulan metode usulan dibandingkan vanilla prompting juga terkonfirmasi melalui skor Helpfulness yang lebih tinggi (3,92 berbanding 3,24) dan validasi Human Evaluation dengan skor kepuasan 5,74 dari skala 6, didukung oleh efisiensi waktu pemrosesan rata-rata satu menit per komponen yang lebih ekonomis dibandingkan pengerjaan manual.

Untuk pengembangan masa depan, disarankan agar arsitektur sistem berevolusi menuju pendekatan Agentic Workflow dengan dukungan pustaka seperti LangGraph guna meningkatkan otonomi dan fleksibilitas orkestrasi antar-modul. Selain itu, kapabilitas analisis statis perlu ditingkatkan agar kompatibel dengan sintaksis Python terkini dan mekanisme impor dinamis, idealnya melalui pengembangan alat analisis dependensi mandiri (custom tool) untuk mengurangi ketergantungan pada pustaka pihak ketiga dan menjamin keberlanjutan sistem jangka panjang .

DAFTAR PUSTAKA

- [1] S. S. Divedi, V. Vijay, S. L. R. Pujari, S. Lodh, and D. Kumar, "A Comparative Analysis of Large Language Models for Code Documentation Generation," in *Proceedings of 1st ACM International Conference on AIware (AIware 2024)*, 2024.
- [2] D. Yang, A. Simoulin, X. Qian, X. Liu, Y. Cao, Z. Teng, and G. Yang, "DocAgent: A Multi-Agent System for Automated Code Documentation Generation," *arXiv preprint arXiv:2504.08725*, 2025.
- [3] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large Language Models: A Survey," *arXiv preprint arXiv:2402.06196*, 2025.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 2017.
- [5] H. Patel and S. Parmar, "Prompt Engineering For Large Language Model," *ResearchGate preprint*, 2024, doi: 10.13140/RG.2.2.11549.93923.
- [6] S. Vatsal and H. Dubey, "A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks," *arXiv preprint arXiv:2407.12994v2*, 2024.
- [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS 2022)*, 2022.
- [8] M. E. Inzaugarat. (2024). "What is a DAG? A Practical Guide with Examples." *DataCamp Blog*. [Online]. Available: <https://www.datacamp.com/blog/what-is-a-dag>

- [9] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab, Stanford, CA, Tech. Rep., 1998.
- [10] R. Tarjan, "Depth-First Search and Linear Graph Algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [11] U. Manber, "Introduction to Algorithms: A Creative Approach," Reading, MA, USA: Addison-Wesley Publishing Company, 1989.
- [12] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008.

Richard Rafer Guy lahir di Sidoarjo, Jawa Timur, Indonesia, pada tahun 2004. Saat ini sedang menempuh studi S1 pada program studi Informatika di Institut Sains dan Teknologi Terpadu Surabaya. Mengambil spesialisasi di bidang Intelligent System. Minat penelitian berfokus pada bidang Kecerdasan Buatan, khususnya dalam penerapan Large Language Models (LLM) untuk pengembangan sistem cerdas

Gunawan adalah seorang dosen di Jurusan Teknik Informatika Institut Sains dan Teknologi Terpadu Surabaya sejak tahun 1991. Ia menyelesaikan studi diploma dan sarjana (S1) Teknik Komputer di Sekolah Tinggi Teknik Surabaya pada tahun 1991, kemudian meraih gelar Master of Computer (S2) dari institusi yang sama pada tahun 2002. Ia meraih gelar Doktor (S3) Teknik Elektro di Institut Teknologi Sepuluh Nopember (ITS) Surabaya pada tahun 2016. Jabatan akademik Guru Besar (Profesor) diperolehnya pada tahun 2024. Penelitiannya berfokus pada bidang Computational Linguistic, Data and Web Mining, dan Information Retrieval. Dengan pengalaman mengajar lebih dari 30 tahun, ia telah memberikan kontribusi signifikan dalam pengembangan ilmu komputer di Indonesia.